Handoff: arXiv Fake-Citation Detection Project

From: R. Wang (rwang@insilicom.com) · **Date:** 2026-08-18 **One-line summary:** We built and validated a 3-channel citation-fabrication detector, ran it over ~1.7M distinct arXiv papers (2007–2026), hand-verified 1,900+ flagged citations, and established: **no Zhao-scale epidemic, but a real, replicated, era-specific LLM signal — ~1 in 1,000 references in 2023–25 arXiv preprints is fabricated (~1 in 550–800 by 2025–26), concentrated in CS, with 39 verified specimens including fabrications already propagating through Google Scholar and into other papers.**

---

1. The scientific state of play

Settled questions

Question	Answer	Evidence
Zhao et al. (arXiv:2605.07723) claim of +0.39pp post-ChatGPT excess	**Rejected**	DOI-bearing channel (9.55M refs): excess sign-flips across baseline forms, ≤+0.23pp under every form
Does LLM-era fabrication exist at all?	**Yes — 39 verified specimens**	2.5% of refined deep candidates [CI 1.7–3.7%]; ~5,400 in v8's 2023–25 pool (CI 3,700–8,000)
Is it era-specific (LLM-caused)?	**Yes, $p \approx 5\times10^{-4}$, directly verified**	0/300 refined pre-LLM (2019–21) controls fabricated vs 2.5% LLM-era
Is it accelerating?	**Yes: ~4× jump in 2025, sustained in 2026H1**	verified fractions 1.4/1.2/5.3/4.1% for 2023/24/25/26H1 ($p \approx 2\times10^{-4}$)
Which fields?	**CS (+17% candidate rate); physics flat; math/stat declining**	per-field table, `_speedtest/fieldrates/run.out`. Fields act as internal control
Is the 23% "not-found" rate fabrication?	**No** — 72% untitled physics refs, 8% parse junk, rest coverage gaps	full 1.86M decomposition

Headline framing

- 2023–25 average: **~0.1%/ref (1 in 1,000)**; 2025–26 current: **~1 in 550–800 and holding**
- ~25× Lancet's biomedical rate (0.0042%), ~3–4× below Zhao's claim
- Undercounts the *published* literature: 15/68 ground-truth fabs existed only in camera-ready versions, invisible to arXiv sampling

Verified specimen catalog (39 items)

In workflow journals (see §7) and `arxiv_sweep_v7/true_fab_residual.tsv` etc. Taxonomy: real-authors+invented-paper (dominant), venue mashups, invented journals, fake-title-on-real-DOI (identifier hijack), impossible venues. **Two propagation cases:** a fake title with a phantom Google Scholar entry (2 citations), and a phantom citation repeated verbatim by a second 2026 paper.

---

2. Infrastructure map

Thing	Where	Notes
Project scripts	`galaxy4:/space/rwang/fake-citation-detector/scripts/`	everything below is relative to this
arXiv TEI corpus (GROBID-extracted)	`/space/eric/citation_data/arxiv/tei/new/<YYMM>.tar.gz`	0704→2606, members `./<id>vN.tei.xml`. Eric owns it
OpenAlex local index (486M works)	`/space/rwang/oa_index/oa_index.db` (149GB sqlite)	table `oa`; indexes on doi/title_norm/(venue,yr,vol). **`author1` = first TOKEN of display name (often a given name!) — root cause of several bugs, see §6**
OpenAlex fuzzy FTS index	`/space/rwang/oa_index/oa_fts.db` (76GB)	484M title_norms; LABELING/refinement only, never the detector
Crossref biblio index	`~/crossref/biblio_index.db` (19GB)	(journal_norm,yr,vol)+page/author; `author1` here IS a clean surname
Crossref title index	`/space/rwang/crossref/crossref_index.db` (66GB)	exact-title phase; WAL, on slow disk — warm into page cache before big runs
arXiv titles cache	`/space/rwang/arxiv_titles.db`	used by `arxiv_authority.py`
OpenAlex raw snapshot	`/space/donghu/openAlex_data/data/works/` (596GB)	donghu owns; source for index rebuilds
Solr	`galaxy:8983` `openalexWorks` / `openalexWorksCurated`	mostly bypassed now (local index is ~1000× faster); hybrid fallback remains
MongoDB (journal authority)	galaxy3:27017, db `openAlex`	credentials: user `openalex_dev_user`, authSource=openAlex; password has an `@` — URL-encode it. Get it from the boss; do not commit it anywhere
Run outputs	`arxiv_sweep_v7/`, `arxiv_sweep_v8/`, `arxiv_sweep_v8b/` in scripts dir	per-month CSVs + per-ref `refs_*.jsonl` (the jsonl is the real data)
Analysis outputs	`/space/rwang/_speedtest/`	v8_analysis.out, v8b_analysis.out, robust1.out, fieldrates/, hijack/, deep_fab_pool_*.jsonl, gold_verify.tsv, fc_verdicts.json
Ground-truth datasets	local + box copies	GPTZero NeurIPS CSV, GPTZero ICLR CSV, CiteTracer structured JSON (`_speedtest/citetracer_structured.json`

Standing directive from the boss: the detector uses EXACT metadata matching only — no fuzzy title search in the detection path. Fuzzy (FTS) is allowed only for labeling/refinement/audit.

3. Pipeline anatomy (what runs what)

```
arxiv_sweep.py --tei-source <TEI> --start YYMM --end YYMM --k K --workers 1
  └─ sample_tei()             samples K v1 papers/month (seed=int(month), k-adaptive cap)
  └─ batch_verify_years.py
      parse_tei_refs()        TEI → ref objects (title/journal/vol/page/author/doi/raw)
      verify_refs()           match phases: DOI → local biblio metadata → oa_local metadata
                              (title-corroborated) → exact-title (crossref_index + oa) → …
      per_ref rows now carry:  found, method, cited_year, has_doi/has_title, mismatch,
                              issue_types, raw/ref_*/matched_* fields, AND (new):
                              not_found_reason, author_hijack, fab_flag   ← ref_classify.py
  └─ outputs months_S_E_kK.csv + refs_S_E_kK.jsonl
```

Directory layout: `scripts/` = the 30 live modules; `scripts/tools/` = index builders (biblio, FAISS, journal authority); `scripts/archive/` = 79 superseded one-off analyses and applied patch scripts, kept for the paper trail (their results live in the report and the memory notebook).

Key modules: - `integrated_lookup.py` — matching backends (oa_local first, Solr fallback) - `oa_local.py` — sqlite backend; `OA_LOCAL_INDEX` env points it at tmpfs copy for big runs - `ref_classify.py` — the classification layers (see §5) - `refine_fc.py` — stage-2 refiner for fab candidates (`validate` / `pool` modes; `Y0/Y1/POOLTAG/RSHARD` envs) - `arxiv_authority.py` — arXiv ID → title; **local-first (cache → oa_index doi 10.48550/arxiv. → API only if `ARXIV_AUTHORITY_API=1`)** — never let the API path run in bulk (see §6) - Analysis: `v8_analyze.py` / `v8b_analyze.py` (5-channel), `robust1.py` (lag-control + flag dumps), `field_rates.py` (per-field), `v7_fakecount*.py`, `neurips_test.py`, `validate_upgrade.py`, `citetracer_recall.py` (ground-truth recall tests)

---

4. How to run a big sweep (the fast config — hard-won)

1. **Copy the two hot indexes into tmpfs** (RAM-backed, pinned): `oa_index.db` + `biblio_index.db` → `/dev/shm/` (168GB). Set `OA_LOCAL_INDEX=/dev/shm/oa_index.db BIBLIO_DB=/dev/shm/biblio_index.db`.
2. **Warm the 66GB `crossref_index.db` into page cache** (`cat … > /dev/null`) — page cache is reclaimable, tmpfs is not; do NOT tmpfs more than the two proven files (see §6 OOM incident).
3. **Shard by month ranges into N single-worker processes** (`run_v8_sharded.sh` pattern). Python's GIL caps one process at ~2 cores regardless of `--workers`; single-threaded shards avoid GIL thrash. **≤16 shards on this shared box.**
4. Everything is resumable per-month (skips existing `months_*.csv`); launch detached (`setsid … < /dev/null &`) so ssh drops can't kill it.
5. Watch `free -g` (available RAM) and load after launch; back off if available drops fast. Reference paces: v8 (1.07M papers) ≈ 20h incl. incidents; v8b (934k) ≈ overnight.

Verification fleets (LLM web-search agents in Claude Code workflows): batch candidates 10/agent, expect session-limit pauses — workflows resume from cache with `resumeFromRunId`. ~600–1,000 items/day is realistic.

---

5. The detector's three channels + classification (validated numbers)

Channel	Catches	Validated performance
`not_found` → `not_found_reason`	invented works	classifier routes fabs to `fab_candidate` tier; tier is ~2.5% true-fab after refinement (LLM era)
`author_hijack` (found refs)	real title + invented authors	recall 57%, FPR 6.9% (GPTZero ground truth) — flags-for-review
`title_hijack` (found refs)	real DOI/ID + invented title	verified true-fraction ~1.7% of flags
combined `fab_flag`	any of the above	**overall recall: 85% (NeurIPS-68), 95.3% (CiteTracer-807)**

`not_found_reason` values: `no_title` (physics-style, can't be LLM-fab), `parse_junk`, `non_article`, `foreign_language`, `datacite_preprint`, `short_title`, `fab_candidate` (the only tier where fabrication hides — still ~97.5% coverage noise before refinement).

Refiner (`refine_fc.py`): clears candidates only on repair-match, extended non-article patterns, or **fuzzy-match-WITH-author-agreement.** Never loosen the author gate — see §6.

---

6. Hard-won lessons (read before touching anything)

1. **The box is shared (donghu's jobs + Solr).** tmpfs pins RAM permanently. We once put 76GB extra in `/dev/shm` while running 30 shards and made the box unreachable for everyone. Budget: the two proven indexes only, ≤16 shards, delete tmpfs copies when done. If the box goes unreachable: arm ONE long-timeout ssh that frees tmpfs on connect; don't spam.
2. **Fuzzy existence checks CLEAR Frankenstein fabrications.** A fabricated citation that recombines a real paper (real title, fake authors — the dominant type) fuzzy-matches its real lookalike and gets waved through. Any "does something similar exist?" logic must require **author agreement** before clearing. This trap produced months of false "0 fabrications" results.
3. **`arxiv_authority._fetch_api` must never run in bulk.** arXiv's API throttles brutally; at workers=1 it serialized entire v8 shards into multi-hour hangs. It's now local-first with the API behind `ARXIV_AUTHORITY_API=1`. Diagnose wedged shards with a `faulthandler.dump_traceback_later` harness + `/proc/<pid>/io` and cpu-tick sampling (wedged = 0 ticks; slow = low-but-nonzero).
4. **`oa_index.oa.author1` is the first token of the display name** (a given name for Western names, accent-mangled: 'stner' for Kästner). Any author comparison must check tokens against the citation's whole raw string, gate initials-style and surname-first citation formats as unjudgeable, and distrust short tokens. A proper surname rebuild from the snapshot is the single highest-value pending improvement.
5. **`pkill -f <script>.py` over ssh kills your own shell** (the pattern matches your own command line). Kill by PID via `ps -eo pid=,cmd= | grep "[b]racket-trick"`.
6. **ssh to galaxy4 drops constantly** (exit 255 / broken pipe mid-command). Anything long-running goes in a detached on-box script writing a flag file; never rely on a tethered ssh loop. Compound remote commands can die halfway — verify state after every drop.
7. **Baseline functional form decides sign.** Never report a post-LLM "excess" from one baseline; fit flat/linear/exp-to-floor and report all three. Sign-flip across forms = artifact (this is the project's pre-registered robustness rule, and it's what killed Zhao's number).
8. **Sampling math bites.** K=1000/month sees ~3% of papers; hand-verifying 0.3% of a bucket finds nothing even when thousands of fabs exist. Compute expected-event counts before concluding anything from a zero.

9. GROBID junk leaks everywhere: body text, LLM prompt fragments, affiliations, figure captions parsed as "references." ~8% of not-founds. The `parse_junk` patterns catch most; expect stragglers in any hand sample.

---

7. Where the verified evidence lives

- **Workflow journals** (this machine): `~/.claude/projects/-Users-Eseer-1st-laptop/<session>/subagents/workflows/wf_*/journal.jsonl` — per-agent verdicts with evidence notes for all 1,900+ verified citations. Key runs: `wf_57ddd07b-337` (the 1,000-item deep verify, 25 fabs), `wf_35b34c50-413` (300 controls + 300×2026, 12 fabs), `wf_27442022-1e4` (150 w/ pre-LLM control), `wf_5241720e-258` (title-hijack 60).
- Samples + labels on the box: `_speedtest/deep_verify_1000.json`, `ctrl2026_verify.json`, `fc_verdicts.json`, `th_verify_sample.json`.
- Full memory/lab-notebook of the project: `~/.claude/projects/-Users-Eseer-1st-laptop/memory/arxiv_replication.md` (chronological, includes every dead end and why).

---

8. Open work, ranked by value

1. **Human audit of the 39 specimens** (an afternoon) + duplicate-verification subsample ➝ hardens the load-bearing 2.5% against "LLM agents verified it" objections.
2. **Surname index rebuild** (overnight, gentle, from donghu's snapshot) ➝ detector recall 95➝~97%, hijack judgeability 55➝90%, better refiner clearing. Then re-validate with `validate_upgrade.py` + `citetracer_recall.py`.
3. **Verify the author-hijack channel's true fraction** (like title-hijack) ➝ tightens the ÷0.64 coverage correction, the loosest number in the total count.
4. **Stage-3 auto-verifier**: productize what the fleets do — S2/DBLP/Crossref/OpenAlex API sweep with author cross-check + cheap-LLM adjudication of the residual. Semantic Scholar API key was pending (ask the boss). This turns candidate pools into verified lists automatically.
5. **CITADEL cross-validation**: Maxim Topaz (Columbia, maxtopaz.com/citadel) has 4,406 verified fabs in PubMed; email was pending. External recall benchmark + biomed comparison.
6. **Per-field verified rates** (need ~100+ specimens per field — bigger verification budget).
7. **2026-dip corpus-edge investigation** (uniform across fields ⇒ instrument; find the mechanism before anyone reads it as behavior).
8. Code cleanup (duplicate/dead code was partially removed; `.bak_*` files document every change).

9. Writeup skeleton (nothing drafted yet)

Methods (corpus ➝ detector ➝ validation ➝ refinement ➝ verification), the five results (Zhao rejection, existence+rate, era-specificity, acceleration, field concentration), limitations (see the memory file — camera-ready blind spot, content-recombination, agent verification, lag-control scope), 39-specimen appendix. The two propagation cases are the hook.

Good luck — the infrastructure is in good shape, the result is real, and the remaining work is hardening, not discovery.